Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 9

Recap and outline

- Wrapper methods
- Re-training
- Backward selection/LOCO
- MNIST image data example
- Feature importance and feature selection

Wrapper methods

$$\widehat{S^*} \in \arg \min_{S \subseteq \{1,2,...,p\}, |S| \leq k} \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(Y^{(i)}, f(X_S^{(i)}))$$

- Given a class of prediction models/functions f, for each $S \subset \{1, 2, ..., p\}$ we solve find the size of the objective
- Applies to any function class $\mathcal{F}$ (later we focus on a neural networks approach)
- Huge computational challenge, need to evaluate every loss function $\mathcal{L}(.,.)$ for 2^p or $\binom{p}{s}$ subsets

Optimization challenges

- Traverse the space of $\binom{p}{s}$ or 2^p in clever ways
- Evaluate each feature individually (LOCO (leave-one-covariate-out)
- Start with an empty set and *greedily* add features (forward selection)
- Start with a full subset and remove features (feature elimination/backward selection)

Re-training

$$\mathbb{E}[Y|X = x] = f_\theta(x)$$

where θ is a set of parameters defining prediction function

- Examples (θ could be regression co-efficients, weights in neural networks, e.t.c.)
- When feature subset changes, θ also re-trained in order to optimize loss/performance
- Need to a learn a different prediction function/parameter set for each subset S

One approach: LOCO (leave-one-covariate-out)

- Leave-one-covariate out (LOCO) is method for estimating *feature importance*

$$I_j = \mathbb{E}[\mathcal{L}(Y, f_{\tilde{\theta}_j}(X_{-j})) - \mathcal{L}(Y, f_\theta(X))]$$

  where X_{-j} denotes the set of features with X_j removed
- Empirical importance

$$\widehat{I_j} = \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}(Y^{(i)}, f_{\tilde{\theta}_j}(X_{-j}^{(i)})) - \mathcal{L}(Y^{(i)}, f_\theta(X^{(i)}))$$

- $\tilde{\theta}_j$ denotes the *re-trained* parameter(s) after X_j is removed
- $\mathcal{L}(.,.)$ is loss function (often mean-squared error $\mathcal{L}(Y, f_\theta(X))) = (Y - f_\theta(X))^2$

LOCO on simple linear model example

$$Y = X_1 + \frac{1}{2}X_2$$

but the features are perfectly correlated:

$$X_2 = 2X_1$$

For any reasonable choice of $\mathcal{L}(.)$

$$I_1 = I_2 = 0$$

Neither feature is important. Selected feature set empty.

$$I_S = \mathbb{E}[\mathcal{L}(Y, f_{\tilde{\theta}_S}(X_{-S})) - \mathcal{L}(Y, f_\theta(X))]$$

However:

$$I_{1,2} > 0$$

As a subset they are important

Issue with identifiability and LOCO

- For highly/perfectly correlated features, LOCO will tend to set them all to have 0 importance
- You can always replace one feature with another (will see in MNIST example)
- For LOCO to be complete, we would need to consider all subsets
- From a conditional independence perspective:

$$I_j = 0 \Leftrightarrow Y \perp X_j | X_{-j}$$

and for any $S \subset \{1, 2, ..., p\}$

$$I_S = 0 \Leftrightarrow Y \perp X_S | X_{S^c}$$

Summarizing non-identifiable example

$$Y = X_1 + \frac{1}{2}X_2$$

but the features are perfectly correlated:

$$X_2 = 2X_1$$

- For this simple model, different methods give completely different results
- OLS/ridge give $S = \{1, 2\}$
- Lasso gives $S = \{2\}$
- LOCO gives $S = \emptyset$

Underlying truth:

$$Y \perp X_1 | X_2, \; Y \perp X_2 | X_1, \; Y \not\perp \{X_1, X_2\}$$

There are two *minimal* subsets, $\{1\}$ or $\{2\}$

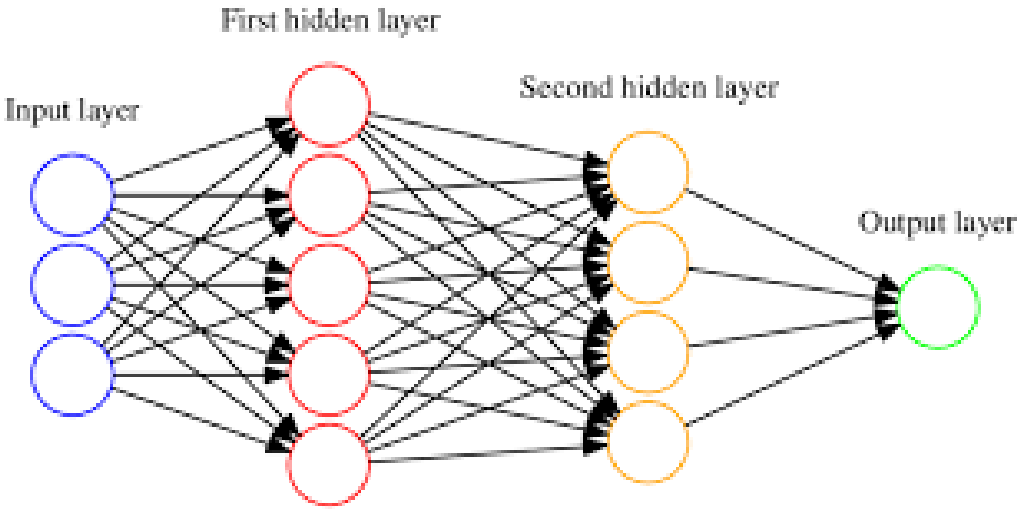
Example: MNIST image data

- Nowadays, large-scale imaging data is becoming more and more prevalent
- MNIST is one of the most widely used test set for images
- Images are 28×28 pixels. Many thousands of images
- **Task:** Prediction/classification: Given noisy images of digits, predict/classify the digits

Neural networks basics

- Imaging (like language) a place where neural networks have really taken off
- Ideal in settings with (i) lots and lots of data; (ii) complex, non-linear structures; (iii) data is difficult to put into a table
- Neural networks have significant expressive powers (many universality results saying that they can fit *any* function, obviously under conditions)
- Many different architectures (CNNs, RNNs, attention-based (LLMs), LSTMs, GNNs, e.t.c.)
- Highly over-parameterized models (θ very high-dimnensional)

Multi-layer perceptrons visual



- Simplest feed-forward neural network. Defined by number of layers L, and number of hidden nodes within each layer H

Multi-layer perceptrons functional

- Prediction function $f_\theta(.)$ Function defined recursively

$$Y = \Phi_L(W_L^T \Phi_{L-1}(W_{L-1}^T \Phi_{L-1}(...W_1^T \Phi_1(X))))$$

- Where $W_1 \in \mathbb{R}^{H \times p}$, $W_L \in \mathbb{R}^{H \times 1}$, $W_\ell \in \mathbb{R}^{H \times H}$ for $\ell \in \{2, ..., L-1\}$ is the set of weight matrices leaned by algorithm (our θ)
- Φ_ℓ is a vectorized version of a univariate activation function being applied at each entry (could be linear, sigmoid, ReLu, e.t.c.)
- Weights often learned using back propagation or more recently gradient descent

MLPs vs Logistic regression for MNIST data

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(max_iter=1000, solver='lbfgs', multi_class='multinomial')
logreg.fit(X_train, y_train)
y_pred_log = logreg.predict(X_test)
acc_log = accuracy_score(y_test, y_pred_log)
print("Logistic Regression accuracy:", acc_log)
```

```
/Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9/site-packages/sklearn/linear_mod
eWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on,
mial'. Leave it to its default value to avoid this warning.
  warnings.warn(
Logistic Regression accuracy: 0.9259
```

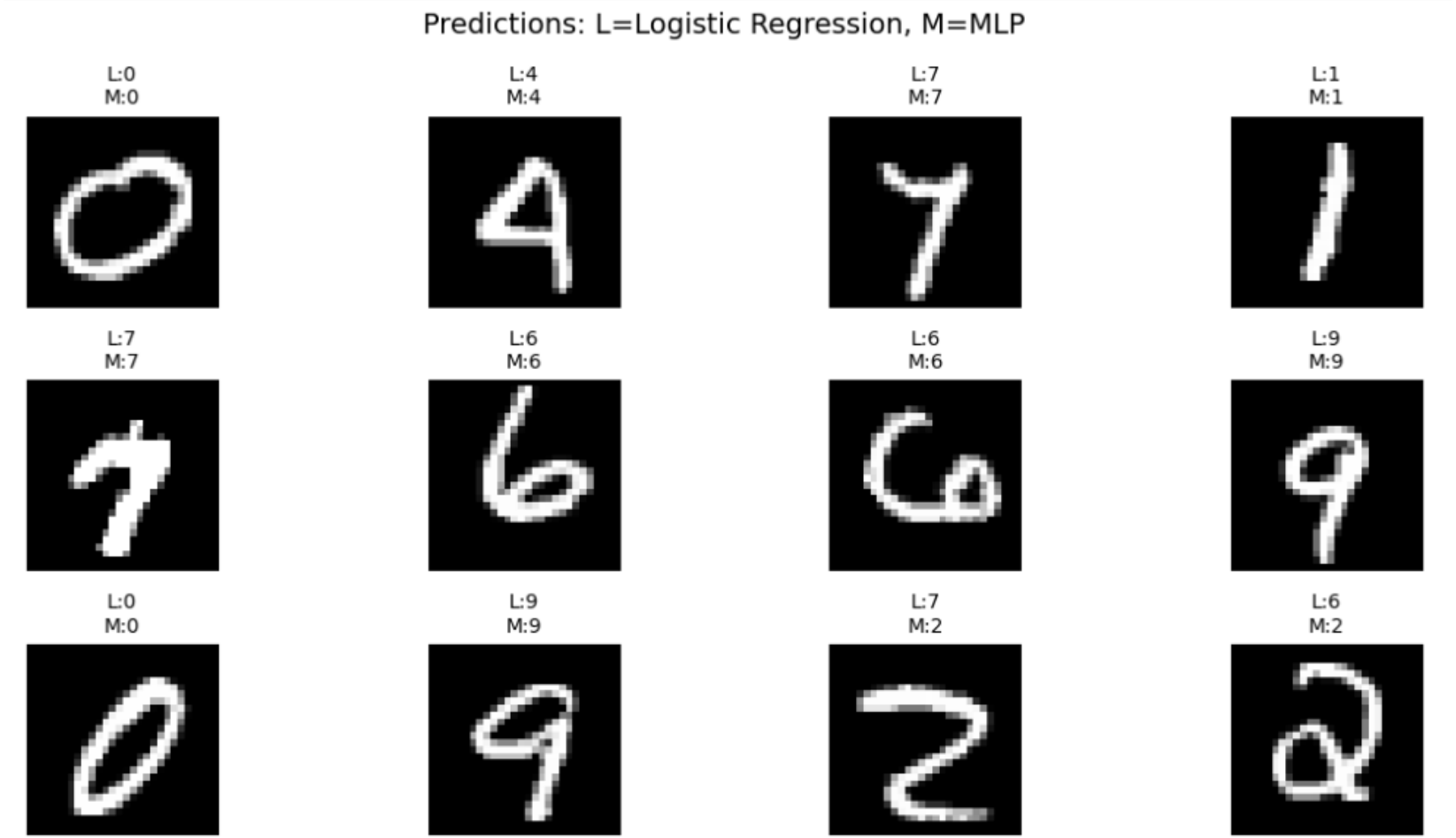
```
# 5. MLP
mlp = MLPClassifier(hidden_layer_sizes=(128,), activation='relu', max_iter=20, random_state=42)
mlp.fit(X_train, y_train)
y_pred_mlp = mlp.predict(X_test)
acc_mlp = accuracy_score(y_test, y_pred_mlp)
print("MLP accuracy:", acc_mlp)
```

```
MLP accuracy: 0.9777
```

MLPs vs Logistic regression image predictions

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

n_images = 12  # number of test images to show
indices = np.random.choice(len(X_test), n_images, replace=False)

fig, axes = plt.subplots(3, n_images//3, figsize=(12,6))
axes = axes.flatten()

for i, idx in enumerate(indices):
    img = X_test[idx].reshape(28,28)
    axes[i].imshow(img, cmap='gray')
    axes[i].axis('off')
    axes[i].set_title(f"L:{y_pred_log[idx]}\nM:{y_pred_mlp[idx]}", fontsize=10)

plt.suptitle("Predictions: L=Logistic Regression, M=MLP", fontsize=14)
plt.tight_layout()
plt.show()
```

MLPs vs Logistic regression image



Logistic regression vs neural networks for image data

- Logistic regression takes a pretty simple pixel-by-pixel approach with a linear relationship
- So loop in the last image image looks kind of like the loop of a ’6’ at a pixel level but the more complex structure of going in the wrong direction is missed

Advantages of neural networks-based approach(es)

- Often viewed as learning more complex features through the hidden nodes (e.g. shape, edge/non-edge, symmetry, e.t.c.)
- With large datasets, can learn complex relationships between features and responses
- Gradient descent-based algorithms lead to reasonably fast implementations
- **But**, often requires extremely large datasets, and if data fits cleanly into a compact table, usually less predictive than linear models/tree ensembles

LOCO for neural networks/MLPs

Recall that LOCO is defined as:

$$\widehat{I_j} = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(Y^{(i)}, f_{\tilde{\theta}_{/j}}(X^{(i)}_{-j})) - \mathcal{L}(Y^{(i)}, f_{\theta}(X^{(i)})).$$

- In the case of MLPs, we trained full model, with learned parameter θ, then need to re-train to learn parameter $\tilde{\theta}_{/j}$.
- Need to re-train p neural networks/MLPs to learn feature importance for p features

Baseline

- We begin by taking an image patch 10×10 and removing from the corner and center of the image and see how the overall performance changes

```python
def train_model(Xtr, ytr):
    model = MLPClassifier(
        hidden_layer_sizes=(128,),
        solver='adam',
        max_iter=10,
        random_state=0
    )
    model.fit(Xtr, ytr)
    return model
```

```python
baseline_model = train_model(X_train_small, y_train_small)
baseline_acc = accuracy_score(
    y_test_small,
    baseline_model.predict(X_test_small)
)

print("Baseline accuracy:", baseline_acc)
```

```
Baseline accuracy: 0.9195
```

Corner

```python
mask_corner = np.ones(28*28)

for i in range(0, 10):
    for j in range(0, 10):
        mask_corner[i*28 + j] = 0

X_train_corner = X_train_small * mask_corner
X_test_corner  = X_test_small * mask_corner

model_corner = train_model(X_train_corner, y_train_small)
acc_corner = accuracy_score(
    y_test_small,
    model_corner.predict(X_test_corner)
)

print("Corner removed accuracy:", acc_corner)
```

```
Corner removed accuracy: 0.921
```

- Removing a corner patch has no significant effect on accuracy $\widehat{I_j} \approx 0$

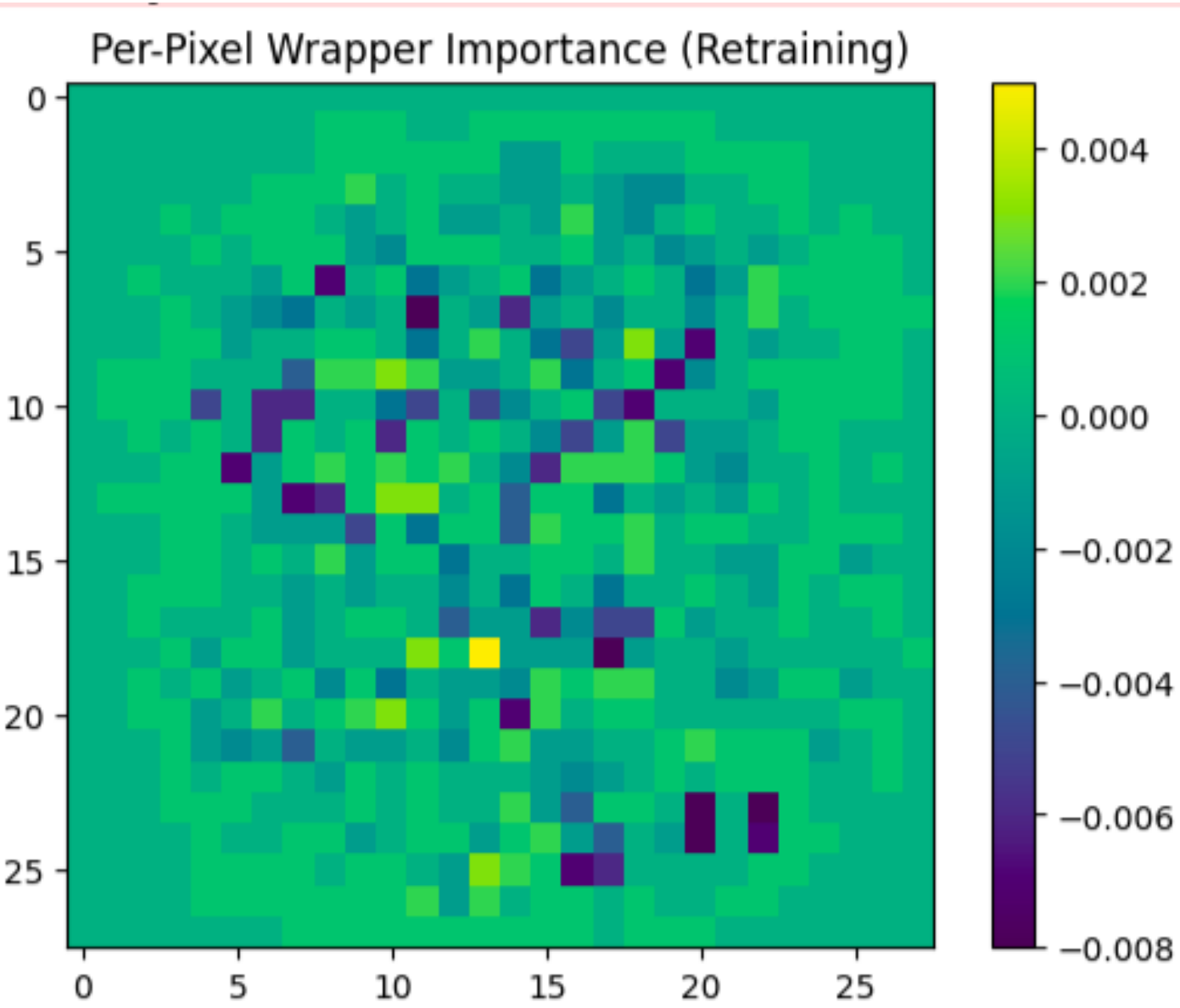
Center

```
mask_center = np.ones(28*28)

for i in range(9, 19):
    for j in range(9, 19):
        mask_center[i*28 + j] = 0

X_train_center = X_train_small * mask_center
X_test_center  = X_test_small * mask_center

model_center = train_model(X_train_center, y_train_small)
acc_center = accuracy_score(
    y_test_small,
    model_center.predict(X_test_center)
)

print("Center removed accuracy:", acc_center)
```

```
Center removed accuracy: 0.841
```

- Removing a center patch has a much more significant impact $\widehat{I_j} \approx 0.08$

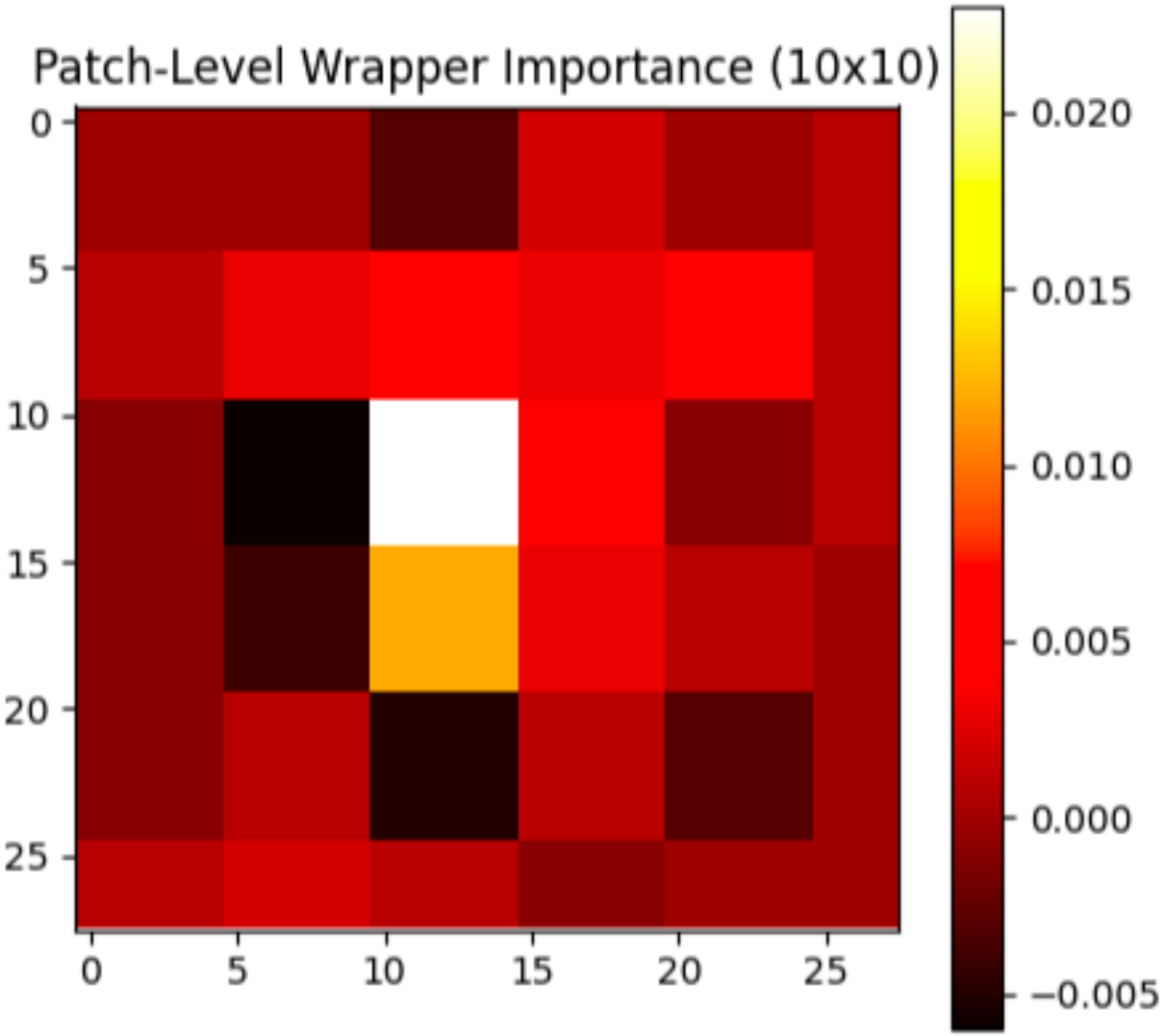
Pixel-level importance image



Issues with pixel-level importance

- Each pixel $p = 28 \times 28 = 784$ treated as a feature
- Individually each pixel has minimal impact because neighboring pixels also carry similar information $\widehat{I_j} \approx 0$ for all j
- Similar to earlier discussion on interchangeable features
- Much better to patch features together
- How we define a *feature* here is crucial
- Next we consider 5×5 patches

Patch-level importance image



Discussion of patch-level importance

- Patch-level importance far more significant
- In particular center-most patch very significant

Feature importance

- I_j refers to feature importance
- Characterizes exactly how much a feature reduces
- For embedded methods, we referred to parameters θ_j (e.g. regression co-efficients, MDI)
- θ_j may also be referred to as feature importance but are intrinsic to the model
- Importantly for LOCO:

$$I_j = 0 \Leftrightarrow Y \perp X_j | X_{-j}$$

(assuming prediction model f is correct)

From feature importance to feature selection

- Feature importance can give a ranking
- How do we go from ranking to selection
- Recall for LOCO:

$$I_j = 0 \Leftrightarrow Y \perp X_j | X_{-j}$$

  (assuming prediction model f is correct)
- From a feature selection perspective, can construct test:

$$\mathcal{H}_0 : I_j = 0 \text{ vs } \mathcal{H}_A : I_j \neq 0$$

Wald-type hypothesis testing result

Recent asymptotic normality result in the literature allows us to perform feature selection. Under $\mathcal{H}_0$

$$\widehat{I_j} \to \text{Normal}(0, \text{Var}(I_j))$$

So using the z-table with appropriate re-scaling we can determine if feature X_j is significant